CSE 4205
Digital Logic Design

Boolean Algebra & Logic Gates

Course Teacher: Md. Hamjajul Ashmafee

Lecturer, CSE, IUT

Email: ashmafee@iut-dhaka.edu

Remaining to Add

- Review other books and internet resources

Basic Definitions

- **Ordinary algebra** – a mathematical system
  - Defined with a set of elements (***S***), a set of operators, and a number of unproved axioms or postulates, where:
    - **i.** **A set of elements, *S*** – any collections of objects having a common property – ***e.g.*** $\{a, b, c \ldots x, y, z\}, \{0,1,2, \ldots\}$
    - **ii.** **A set of operators** – a set of **rules (operations)** that defined on the set ***S*** – ***e.g.*** $\{+,*\}$
    - **iii.** **Postulates** – form of the **basic assumptions** from where other rules, theorems, and properties of the system are deduced

Common Postulates of a Mathematical System

1. **Closure:** A ***set*** is said to satisfy a closure property if it is **closed** under an operation or collection of operations.
   - ***i.e.,*** A ***set*** is closed with respect to a binary operation if performance of that operation on the members of the set always produce a result from the **same** set
   - ***Example:*** The set of natural number, $\boldsymbol{N} = \{1,2,3, \dots\}$ is closed with respect to the binary operator plus (**+**) by the rules of arithmetic addition but not closed with respect to binary operator minus (**-**) by the rules of arithmetic subtraction.
     - Here, $a, b \in \boldsymbol{N}$ and we obtain a unique $c \in \boldsymbol{N}$ by the operation $a + b = c$ but $2 - 3 = -1$ where $2,3 \in \boldsymbol{N}$ but $-1 \notin \boldsymbol{N}$
2. **Associative Law:** A binary operator # on a set $\boldsymbol{S}$ is said to be associative whenever:

$$(\boldsymbol{x} \# \boldsymbol{y}) \# \boldsymbol{z} = \boldsymbol{x} \# (\boldsymbol{y} \# \boldsymbol{z}), \textbf{\textit{ for all }} \boldsymbol{x}, \boldsymbol{y}, \boldsymbol{z} \in \boldsymbol{S}$$

Common Postulates…

3. **Commutative Law:** A binary operator # on a set $\boldsymbol{S}$ is said to be commutative whenever:

$$\boldsymbol{x \# y = y \# x, \text{ for all } x, y \in S}$$

4. **Identity element:** A set $\boldsymbol{S}$ is said to have an identity element with respect to a binary operation # on $\boldsymbol{S}$ if there exists an element $e \in \boldsymbol{S}$ with the property that:

$$\boldsymbol{x \# e = e \# x = x, \text{ for all } x, e \in S}$$

- ***Example:*** The element **0** is an identity element with respect to the binary operation plus (**+**) on the set of integers $\boldsymbol{I} = \{\ldots -3, -2, -1, 0, 1, 2, 3 \ldots\}$ but not on the set ***N*.**
  - Here, $x + 0 = 0 + x = x$, for any $x \in \boldsymbol{I}$

Common Postulates…

5. **Inverse:** A set $\boldsymbol{S}$ having the identity element $\boldsymbol{e}$ with respect to a binary operator $\boldsymbol{\#}$ is said to have an inverse whenever, for every $\boldsymbol{x} \in \boldsymbol{S}$ there exists an inverse element $\boldsymbol{y} \in \boldsymbol{S}$ such that:

$$\boldsymbol{x \# y = e, \text{ for all } x, y, e \in S}$$

   - ***Example:*** In the set ***I***, $\boldsymbol{e = 0}$ with **binary operator *+***, the **inverse** of every element *a* is *(-a)* since $a + (-a) = 0$

6. **Distributive Law:** If **#** and **$** are two binary operators on a set *S*, **#** is said to be distributive over **$** whenever:

$$\boldsymbol{x \# (y \$ z) = (x \# y) \$ (x \# z)}, \textit{ for all } \boldsymbol{x, y, z \in S}$$

Basic Definitions - II

- **Field**: A set of elements, together with any two binary operators, each having **properties 1 to 5** and both operators combined to give **property 6.**
  - **Basically,** a field is an entity that belongs the preceding properties
  - **Implementation** of an algebraic structure
  - ***Example:*** Set of real number, **R**, with two binary operators $+$ and $*$ form the **field of real numbers**. (*verification?*)

Boolean Algebra

- In 1854, **George Boole** introduced systematic treatment of logic and developed Boolean algebra
- In 1938, **C. E. Shannon** materialized a two-level Boolean algebra
  - Also called switching circuit/algebra
  - First introduced the properties of bistable electrical switching circuit
- For formal definition of Boolean algebra, we follow the postulates formulated by **E. V. Huntington** (1904).
- ***Boolean algebra*** is a **field** (implementation of algebraic structure) with a set of elements ***B,*** together with two binary operators (+) and (.) and follows the **Huntington** postulates.

Huntington Postulates for Boolean Algebra

1. **a**. **Closure** with respect to the binary operator +

   b. **Closure** with respect to the binary operator **.**
2. **a**. **An identity element** with respect to +, is designated by 0 :

   $$x + 0 = 0 + x = x$$

   b. **An identity element** with respect to **.**, is designated by 1 :

   $$x.1 = 1.x = x$$
3. **a**. **Commutative** with respect to + : $x + y = y + x$

   b. **Commutative** with respect to **.** : $x.y = y.x$
4. **a**. . is **distributed** over + : $x.(y + z) = (x.y) + (x.z)$

   b. + is **distributed** over . : $x + (y.z) = (x + y).(x + z)$

Huntington Postulates…

5. For every element $x \in B$ there exists an inverse element $x' \in B$ (**complement of x**) such that

$$i. \quad x + x' = 1$$

$$ii. \quad x.x' = 0$$

6. There exists at least **two elements** $x, y \in B$ such that $x \neq y$

Boolean Algebra VS Ordinary Algebra

Field of binary and real numbers

- Huntington postulates do not mention **associative law,** but it holds here. (derivable from other postulates)
- The **distributive law of (+) over (.)** is only valid for Boolean algebra but not for ordinary algebra. [***i.e.,*** $x + (y.z) = (x + y).(x + z)$]
- Boolean algebra does not have any **additive or multiplicative inverse (0 and 1 respectively for ordinary algebra)**.
    - So, there is **no subtraction or division** operations in Boolean algebra
- **Complement** is only available in Boolean algebra
- Two valued Boolean algebra deals with set ***B*** having **two elements 0 and 1** whereas real numbers, ***N*** deals with an infinite set of elements

Two-valued Boolean Algebra

- Two valued Boolean algebra is defined on a set of two elements, B = {0,1} with rules of two binary operators (+ and .)
    - Huntington postulates are valid for the set B and two binary operators
    - Slide # 9, 10 can be verified over those given circumstances
- Application of Boolean algebra:
    - Gate type circuits
    - Development of theorems and properties
- Also known as **switching algebra**, **binary logic**

Property: Duality

- Every algebraic expression deducible from the **postulates of the Boolean algebra** remains **valid** if the operators and identity elements are interchanged.
    - For this reason, Huntington postulates are listed in pairs.
    - In Boolean algebra, elements of B and the identity elements are identical (***i.e.,*** 0 and 1)
- For dual of any algebraic expression, we simply interchange OR and AND operators and replace 1s by 0s and 0s by 1s.
    - **Example:** Postulate – 5 from Huntington postulates:

i. $x + x' = 1$

ii. $x.x' = 0$

Basic Theorems

Postulate 2 (identity)	a. $x + 0 = x$	b. $x.1 = x$
Postulate 3 (commutative)	a. $x + y = y + x$	b. $x.y = y.x$
Postulate 4 (distributive)	a. $x(y + z) = xy + xz$	b. $x + yz = (x + y)(x + z)$
Postulate 5 (complement)	a. $x + x' = 1$	b. $x.x' = 0$
Theorem 1	a. $x + x = x$	b. $x.x = x$
Theorem 2	a. $x + 1 = 1$	b. $x.0 = 0$
Theorem 3 (involution)	$(x')' = x$	
Theorem 4 (associative)	a. $x + (y + z) = (x + y) + z$	b. $x(yz) = (xy)z$
Theorem 5 (De Morgan)	a. $(x + y)' = x'y'$	b. $(xy)' = x' + y'$
Theorem 6 (absorption)	a. $x + xy = x$	b. $x(x + y) = x$

Proof of Basic Theorems

THEOREM 1(a): $x + x = x$.

Statement	Justification
$x + x = (x + x) \cdot 1$	postulate 2(b)
$= (x + x)(x + x')$	5(a)
$= x + xx'$	4(b)
$= x + 0$	5(b)
$= x$	2(a)

Proof of Basic Theorems…

THEOREM 1(b): $x \cdot x = x$.

Statement	Justification
$x \cdot x = xx + 0$	postulate 2(a)
$= xx + xx'$	5(b)
$= x(x + x')$	4(a)
$= x \cdot 1$	5(a)
$= x$	2(b)

Proof of Basic Theorems…

THEOREM 2(a): $x + 1 = 1$.

Statement	Justification
$x + 1 = 1 \cdot (x + 1)$	postulate 2(b)
$= (x + x')(x + 1)$	5(a)
$= x + x' \cdot 1$	4(b)
$= x + x'$	2(b)
$= 1$	5(a)

THEOREM 2(b): $x \cdot 0 = 0$ by duality.

Proof of Basic Theorems…

THEOREM 6(a): $x + xy = x$.

Statement	Justification
$x + xy = x \cdot 1 + xy$	postulate 2(b)
$= x(1 + y)$	4(a)
$= x(y + 1)$	3(a)
$= x \cdot 1$	2(a)
$= x$	2(b)

THEOREM 6(b): $x(x + y) = x$ by duality.

Proof of Basic Theorems…

- **Theorem 3:** $(x')' = x$.
  - From postulate 5 (complement), we have $x + x' = 1$ and $x.x' = 0$ which define the complement of x. So, the complement of x' is x and $(x')'$.
    - As a result, we can get, $x = (x')'$
  - Complement of any set, $\boldsymbol{x}$ is $\boldsymbol{U} - \boldsymbol{x}$
    - Therefore, complement of complement $= \boldsymbol{U} - (\boldsymbol{U} - \boldsymbol{x}) = \boldsymbol{x}$

Truth Table

- Theorems of Boolean algebra can be proven using truth table as well.
  - Both sides of the relation are checked to see whether they produce identical results for all possible combinations of the variables
- The algebraic proofs of the **associative law** and **De Morgan's** theorem are long but can be proved easily with **truth tables**.

Proof using Truth Table

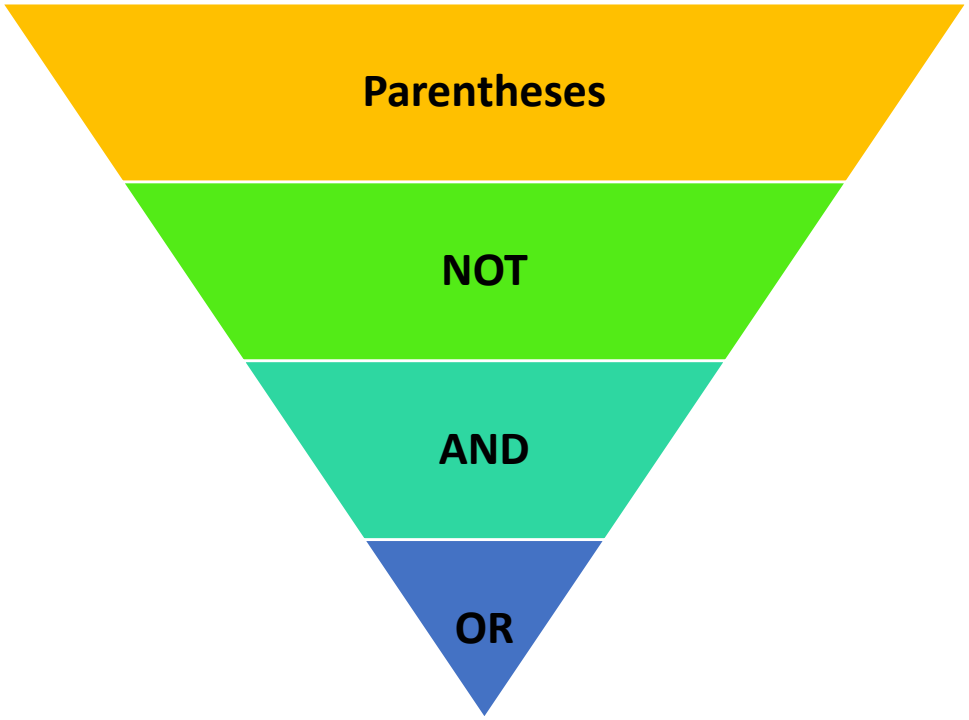
- **Absorption Rule** $(x + xy = x)$:

X	Y	XY	X+XY
0	0	0	0
0	1	0	0
1	0	0	1
1	1	1	1

- **De-Morgan's Law** $[(x + y)' = x'y']$:

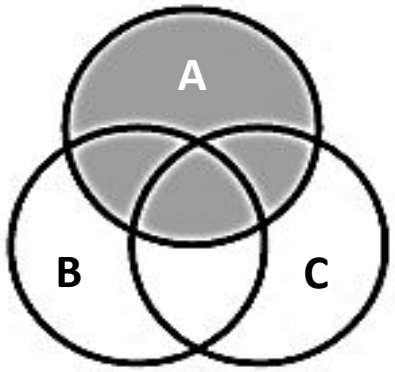
X	Y	X+Y	(X+Y)'	X'	Y'	X'Y'
0	0	0	1	1	1	1
0	1	1	0	1	0	0
1	0	1	0	0	1	0
1	1	1	0	0	0	0

Operator Precedence

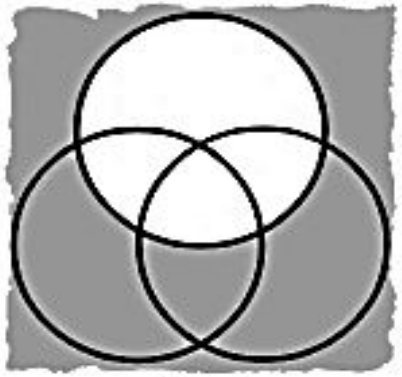


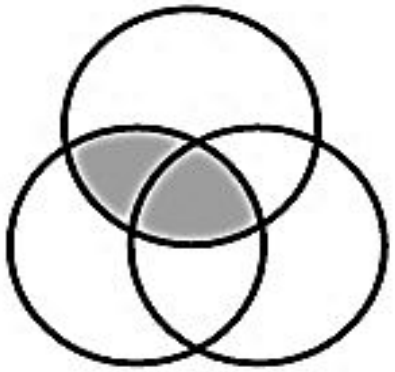
Venn Diagram

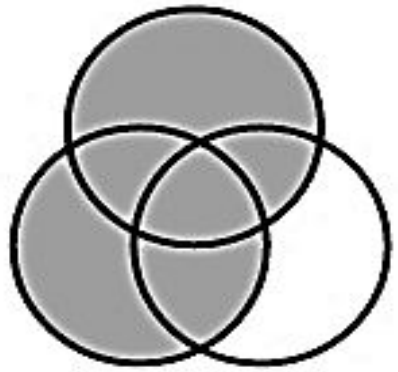
It has similarity with Boolean expressions.

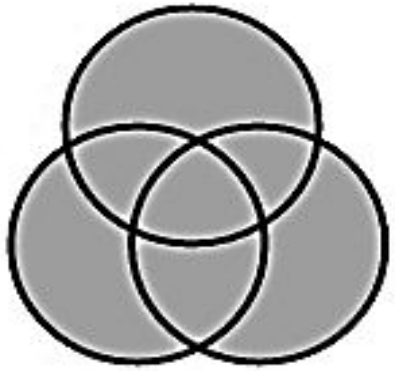


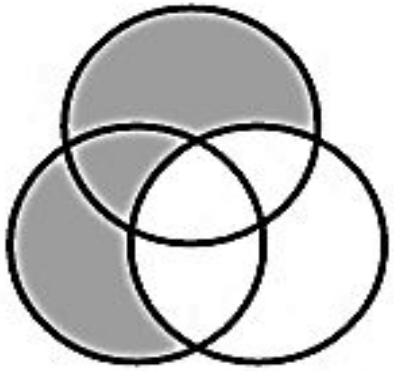
A

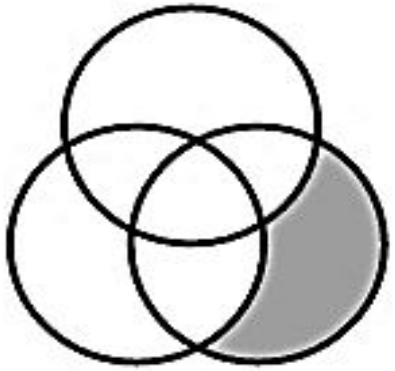

Not A

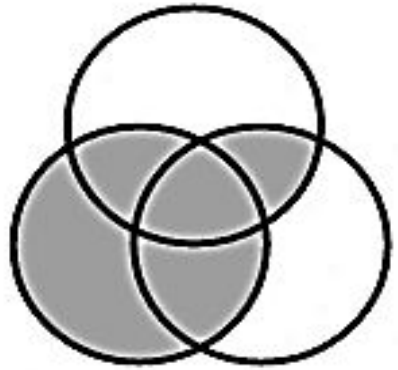

A And B


A Or B


A Or B Or C


(A Or B) And Not C


C And Not A And Not B


B Or (C And A)

Boolean Function

- **Boolean function** is an **algebra** with Boolean variables and logic operations
  - Expressed as an **algebraic expression** with **binary variables**, **Boolean constants (0 and 1)**, **logic operation symbols**
  - For given values for binary variables the **function** produces 0 or 1 as **output**
  - **Example:** $F_1 = x + y'z$
- **Expression** is a mathematical phrase whereas **function** is a logical relationship between a set of **input** variables and corresponding **output**
- Boolean function can be represented with **Truth Table** as well
  - **Number of the rows** of the table is 2^n **where** ***n*** = number of variables in the function
  - **Binary combination** of all variables can be obtained counting from $\mathbf{0} \sim \mathbf{2^n - 1}$ in binary

Boolean Function in Truth Table

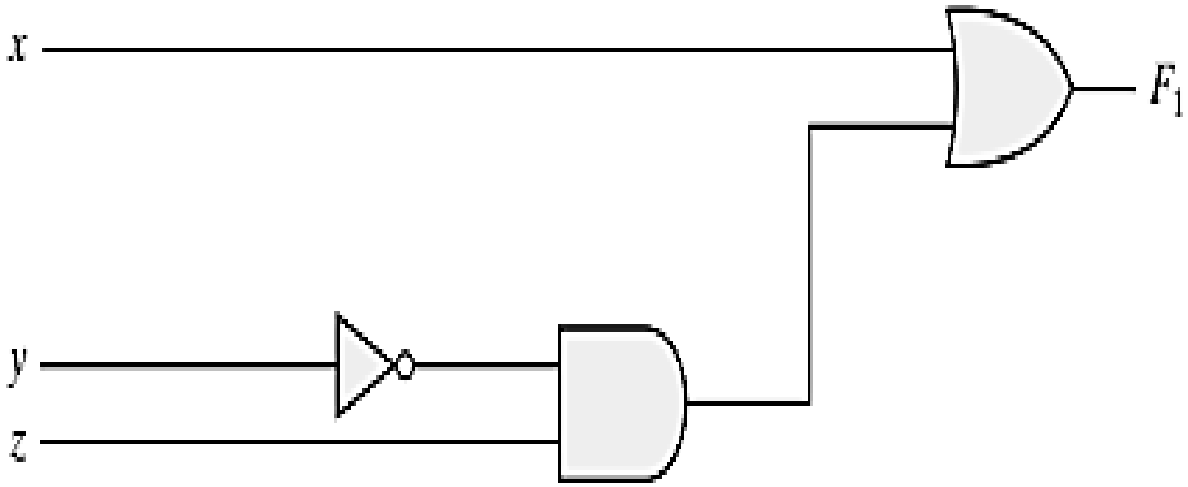
x	y	z	F_1
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

It is obvious from the truth table that, the function is 1 when $(x = 1)$ or ($y = 0$ and $z = 1$)

Boolean Function in Circuit Diagram

- A Boolean function can be expressed with **circuit diagram** composed of logic gates following a particular structure
    - Also known as **schematic diagram**
- Generally, variables in **lowercase** of the function are considered as **input**, and variables in **uppercase** as **output** of the circuit
    - Express the **relationship** between input and output of the circuit
- Instead of listing all possible combinations of input and corresponding output like truth table, it rather indicates **how to generate** the **output logic value** from given **input logic values**.

Boolean Function in Circuit Diagram: Example



Boolean Function: Reformation

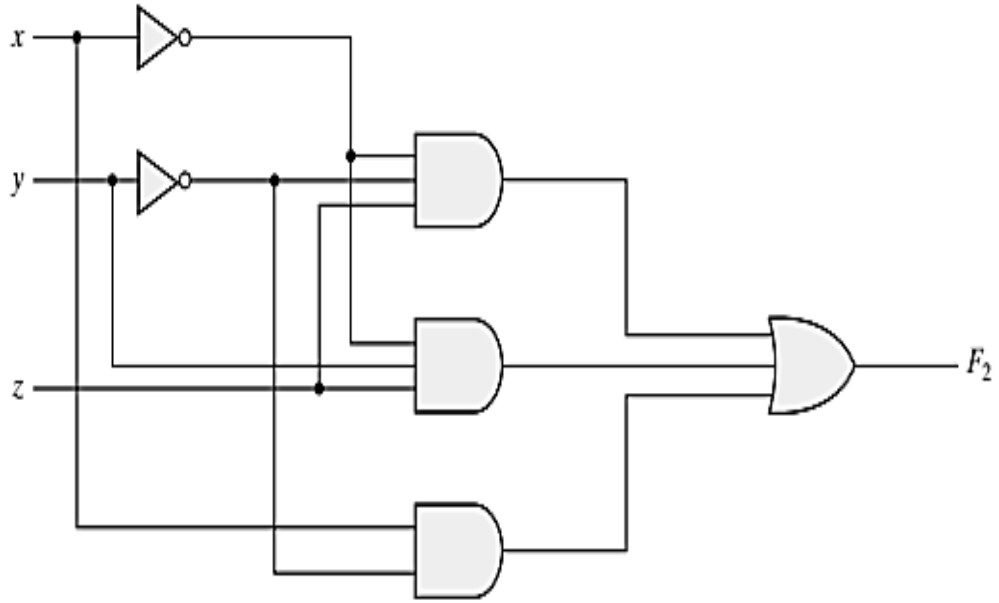
- In algebraic form, a function can be **expressed** in a **variety of ways**, all which of which have **equivalent logic**
  - Also, it can be transformed into **different circuit diagrams**
  - But there is only **one way to express** a Boolean function through the **truth table**
- **Note: Truth table** is the only way to verify the expressions' equivalence
- The **motivation** of the Boolean algebra:
  - Manipulate a Boolean expression using rules of Boolean algebra to reduce it into a simpler expression
  - Thus reduce the number of gates, inputs, and interconnections in the circuit for the same function
  - Ultimately **reduce** the complexity, cost, and **increase** the reliability, efficiency
    - Finding the most economic logic representation is an important design task

Boolean Function: Redesign - Example

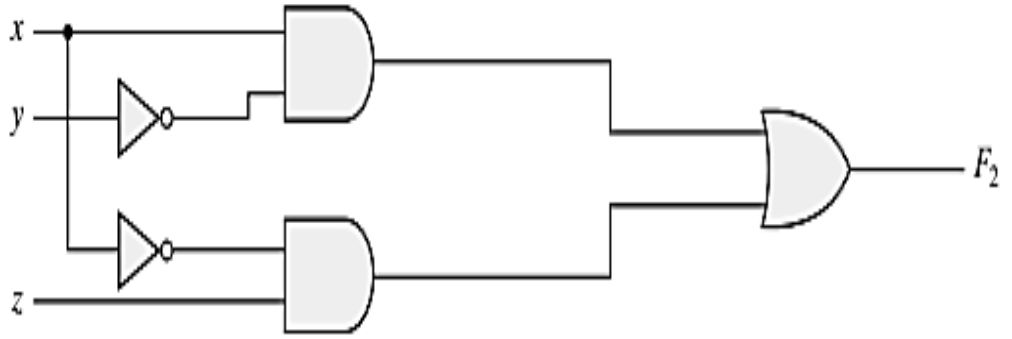
$$F_2 = x'y'z + x'yz + xy'$$

$$F_2 = x'y'z + x'yz + xy' = x'z(y' + y) + xy' = x'z + xy'$$

x	y	z	F_2
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0



(a) $F_2 = x'y'z + x'yz + xy'$



(b) $F_2 = xy' + x'z$

Algebraic Manipulation

- In any Boolean expression –
  - A **Term** (***e.g.,*** xyz') is represented by **a gate with inputs**
  - Each **Literal/Variable** within a term designate **an input** in **primed** or **unprimed** form
- **Example:**

$$F_1 = x'y'z + x'yz + xy' \quad \textbf{(3 terms and 8 literals)}$$

$$F_2 = x'z + xy' \quad \textbf{(2 terms and 4 literals)}$$

- **Task:** To **simplify** the expression, we have to **reduce** the number of terms or number of literals or both (most frequent task in Boolean algebra)
- **Methods**: cut-and-try procedure, K-map, computer minimization programs

Simplification of Boolean Function

- **Cut and Try procedure:**

$$x(x' + y) = xx' + xy = 0 + xy = xy.$$

$$x + x'y = (x + x')(x + y) = 1(x + y) = x + y.$$

$$(x + y)(x + y') = x + xy + xy' + yy' = x(1 + y + y') = x.$$

Complement of a function

- **Complement** of a function *F* is *F'*
  - **Interchanging** the output of function ***F*** produces the ***F'*** (***i.e.*** if $F = 1$, then $F' = 0$ and vice versa) in the **truth table**
- Algebraically, the ***F'*** is derived following **De Morgen's law**
  - **De Morgan's Law:** Interchanging **AND** and **OR** operators and complementing each literal
  - De Morgan's law for any number of variables (**generalized formula**):

$$(A + B + C + D + \cdots + F)' = A'B'C'D' \ldots F'$$

$$(ABCD \ldots F)' = A' + B' + C' + D' + \cdots + F'$$

- **Alternative Method:** Take the dual of the expression and complement each literal.

Complement of a function: Example

$$F_1 = x'yz' + x'y'z$$

$$F_1' = (x'yz' + x'y'z)' = (x'yz')'(x'y'z)' = (x + y' + z)(x + y + z')$$

$$F_2 = x(y'z' + yz)$$

$$F_2' = \ ?$$

Minterms

- Binary Variable (***e.g.,*** **x**)
  - Can be appeared in its either **normal (x)** or **complement (x')** **form**
- **Minterm:**
  - A product (**ANDed term**) that contains **all variables** of a particular function in either complemented (0 = absence of any variable) or non-complemented (1 = presence of any variable) form.
  - Also known as **Standard Product**
  - ***n*** variables only can be combined with **AND** operator(s) to form 2^n minterms
    - Those minterms are numbered in **binary** between $\mathbf{0} \sim \mathbf{2^n - 1}$
  - Symbol of minterms – m_j
    - Where ***j*** = **decimal equivalent** of those binary representations

Minterms

			Minterms	
x	y	z	Term	Designation
0	0	0	$x'y'z'$	m_0
0	0	1	$x'y'z$	m_1
0	1	0	$x'yz'$	m_2
0	1	1	$x'yz$	m_3
1	0	0	$xy'z'$	m_4
1	0	1	$xy'z$	m_5
1	1	0	xyz'	m_6
1	1	1	xyz	m_7

Maxterms

- **Maxterm:**
  - A sum (**ORed term**) that contains all variables of a particular function in either non-complemented (0 = absence of any variable) or complemented (1 = presence of any variable) form.
  - Also known as **Standard Sum**
  - ***n*** variables only can be combined with **OR** operator(s) to form 2^n maxterms.
    - Those maxterms are numbered in **binary** between $\mathbf{0} \sim \mathbf{2^n - 1}$
  - Symbol of maxterm – M_j
    - Where ***j*** = **decimal equivalent** of those binary representations

- ***Note:*** *Maxterm and its corresponding minterm are complements each other*

$$m_j = M_j'$$

Maxterms

			Maxterms	
x	y	z	Term	Designation
0	0	0	$x + y + z$	M_0
0	0	1	$x + y + z'$	M_1
0	1	0	$x + y' + z$	M_2
0	1	1	$x + y' + z'$	M_3
1	0	0	$x' + y + z$	M_4
1	0	1	$x' + y + z'$	M_5
1	1	0	$x' + y' + z$	M_6
1	1	1	$x' + y' + z'$	M_7

Boolean Functions Using Minterms

- **Expression of a Boolean function**
  - From a given *truth table*, minterms are produced from those combinations of the variables which produces 1 (**True**) in the given function and then **OR** all of those minterms.

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$$

$$f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$$

Note: **Any Boolean function can be expressed as a sum of minterms**

Complement of Boolean Functions

- **Procedure using truth table:**
  - Taking each combinations of variables (**minterms**) that produces 0 (**False**) in the function and then OR all of those terms.
    - ***i.e.*** Considering those minterms which are 0 in the truth table

x	y	z	Function f_1	Function f_2
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$f_1' = x'y'z' + x'yz' + x'yz + xy'z + xyz'$$

$f_2' = ?$

Boolean Functions Using Maxterms

- **Procedure**:
  - Taking the complement of complemented function (f_1').

$$f_1 = (x + y + z)(x + y' + z)(x' + y + z')(x' + y' + z)(\boldsymbol{x + y' + z'})$$
$$= M_0 \cdot M_2 \cdot M_3 \cdot M_5 \cdot M_6$$

$$f_2 = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z)$$
$$= M_0 M_1 M_2 M_4$$

- **Note:**
  - Any Boolean function can also be written as a **product (AND) of maxterms**.
  - *Boolean function written as a sum of minterms or product of maxterms* – **Canonical form**

Boolean Function in Canonical Form

Using Minterms

- Usually, Boolean function is expressed in **canonical form** using minterms.
  - Known as **Sum of minterms/products (SOP)**
  - If the function is not in this form,
    - At first, the expression is **expanded** into a sum of AND terms
    - Then, Each term is inspected whether it contains all the variables.
      - If any variable (***e.g.,*** x) is **missed**, that very term is **AND**ed with $(x + x')$.

$$F = A + B'C$$

$$F = A'B'C + AB'C + AB'C + ABC' + ABC$$

$$= m_1 + m_4 + m_5 + m_6 + m_7 \qquad \text{Notation 1}$$

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7) \qquad \text{Notation 2}$$

Boolean Function in Canonical Form…

Using Minterms

- It is also convenient to **derive** minterms of a Boolean function obtaining the **truth table** of the function directly from the expression and selecting the minterms (**=1**) from the truth table.
  - **Example:** $F = A + B'C$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Boolean Function in Canonical Form

Using Maxterms

- Boolean function is also expressed in **canonical form** using maxterms
  - Known as **Product of maxterms/sums (POS)**
  - If the function is not in this form,
    - At first, the expression is **expanded** into a **product** of **OR** terms
      - **Distributive law is more convenient** to use here. [***i.e.,*** $x + yz = (x + y)(x + z)$]
    - Each term is inspected whether it contains all the variables.
      - If any variable (x) is missed, **OR**ed that term with ($x.x'$).

$$F = xy + x'z$$

$$F = (x + y + z)(x + y' + z)(x' + y + z)(x' + y + z')$$

$$= M_0M_2M_4M_5 \qquad \text{Notation 1}$$

$$F(x, y, z) = \Pi(0, 2, 4, 5) \qquad \text{Notation 2}$$

Conversions between Canonical Forms

- The complement of a function expressed as SOP **equals** SOP of missing minterms from the original function

$$F(A, B, C) = \Sigma(1, 4, 5, 6, 7)$$

$$F'(A, B, C) = \Sigma(0, 2, 3) = m_0 + m_2 + m_3$$

- If the complement of F' is taken following **De Morgan's Law**, we will get the original function F in different form

$$F = (m_0 + m_2 + m_3)' = m_0' \cdot m_2' \cdot m_3' = M_0 M_2 M_3 = \Pi(0, 2, 3)$$

- **Form this conversion, it is proved that:**

$$m_j = Mj'$$

Conversions between Canonical Forms…

- **General conversion procedure:**
  - To convert from one canonical form to another, interchange the symbols ∑ and ∏ and list those numbers missing from the original form
    - To find the missing terms, one must realize that the total number of minterms or maxterms is 2^n, where ***n*** is the number of binary variables in the function

Conversions between Canonical Forms…

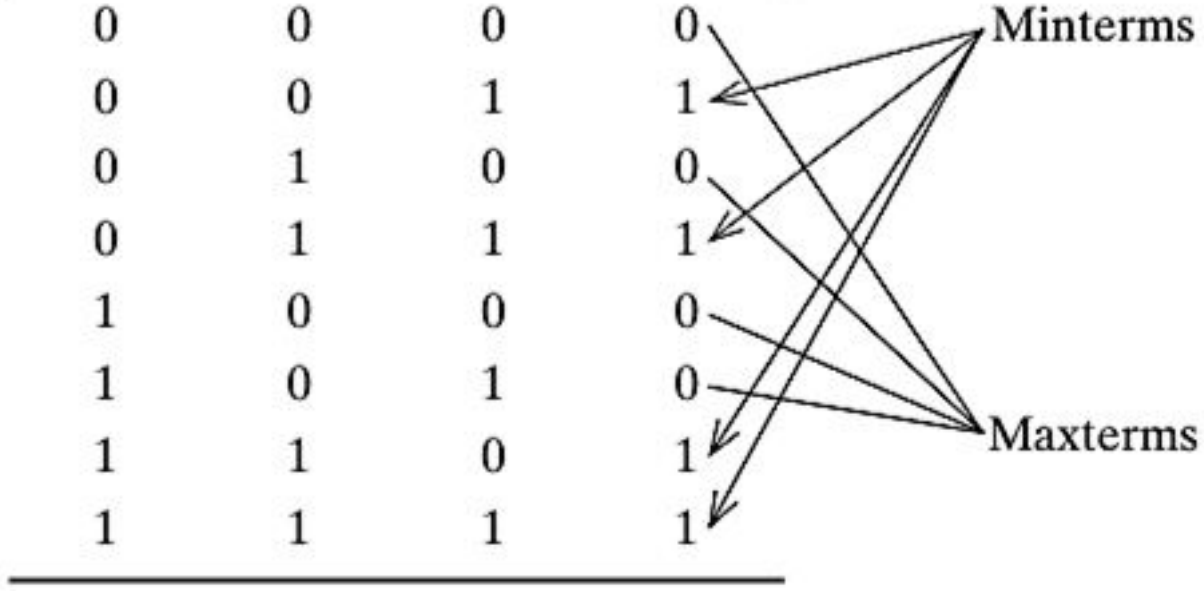
- **Another Example:**

$$F = xy + x'z$$

$$F(x, y, z) = \Sigma(1, 3, 6, 7)$$

$$F(x, y, z) = \Pi(0, 2, 4, 5)$$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1



Note: In the truth table, 1's of the function represent the minterms and 0's represent the maxterm which are true for the given function.

Standard Forms

- **Canonical forms:**
  - Both are easily formed from truth table
  - **Rarely** have least number of literals (!!!)
    - Each minterm or maxterm must contain **all variables**; either primed or unprimed
- **Standard form:**
  - Terms may have **one**, **two**, **three** or **any** number of literals.
  - **Two types:** SOP and POS

Standard Forms: SOP and POS

- **SOP**
  - Has **ANDed terms** (products) which are finally **ORed** (sum)
  - **Logic diagrams** contain a group of **AND** gates followed by a single **OR** gate
    - It's assumed that the complements of variables are directly available in their input
    - Known as **two-level-implementation**

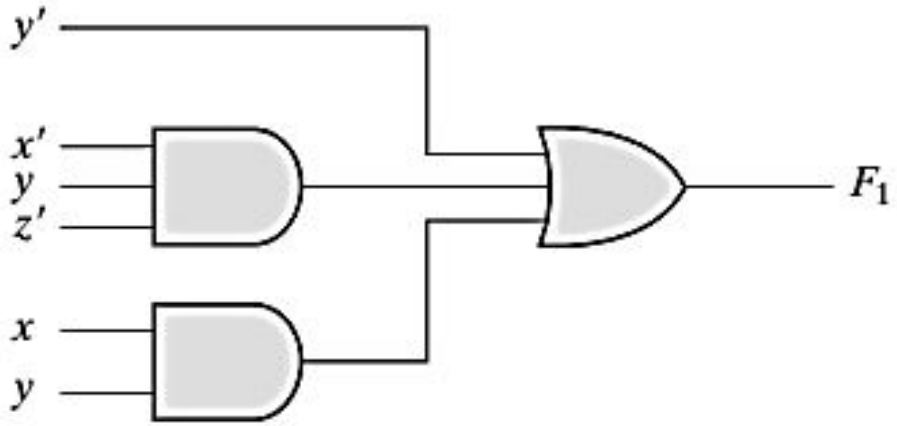
SOP: $F_1 = y' + xy + x'yz'$

- **POS**
  - Has **ORed terms** (sums) which are finally **ANDed** (product)
  - **Logic diagrams** contain a group of **OR** gates followed by a single **AND** gate
    - It's assumed that the complements of variables are directly available in their input
    - Another **two-level-implementation**

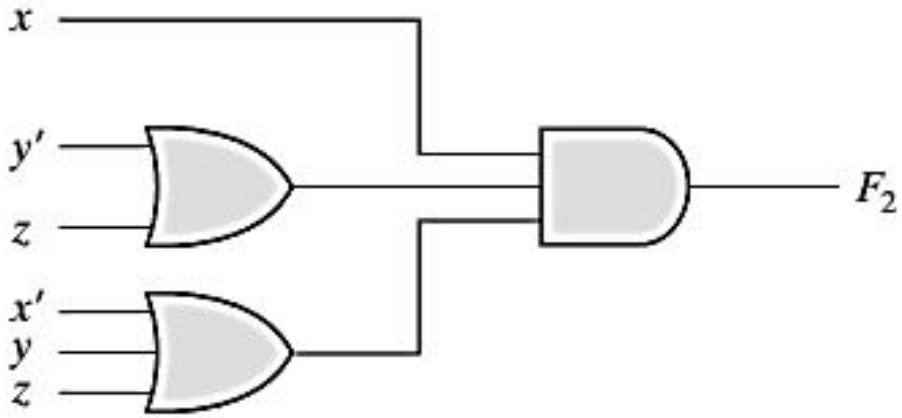
POS: $F_2 = x(y' + z)(x' + y + z')$

SOP and POS: Circuit Diagram

Two Level Implementation



(a) Sum of Products



(b) Product of Sums

Note: **It is assumed that Input variables are directly available in their complements.**

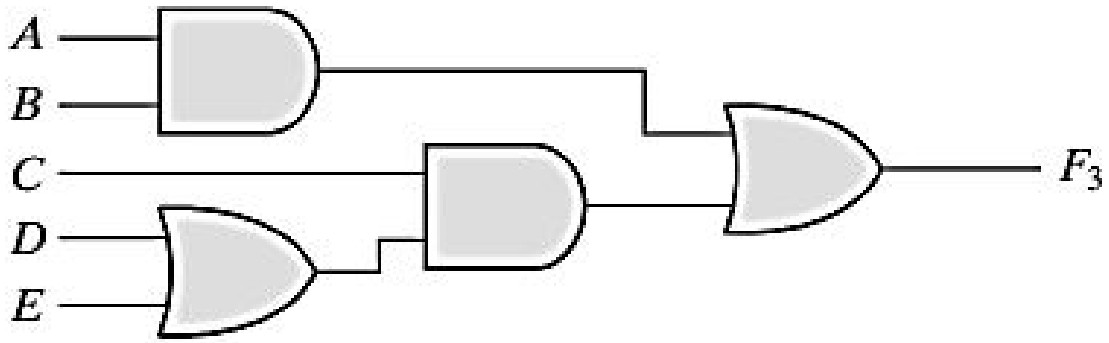
Standard Forms and Nonstandard Forms

- **Nonstandard form:**
  - The Boolean expression is **neither** in **SOP** **nor** in **POS** form
    - May require **random** number of **AND an OR gates**
    - Also, the number of **levels for implementation** are **random**
  - Can be converted to standard form following the **distributive law**

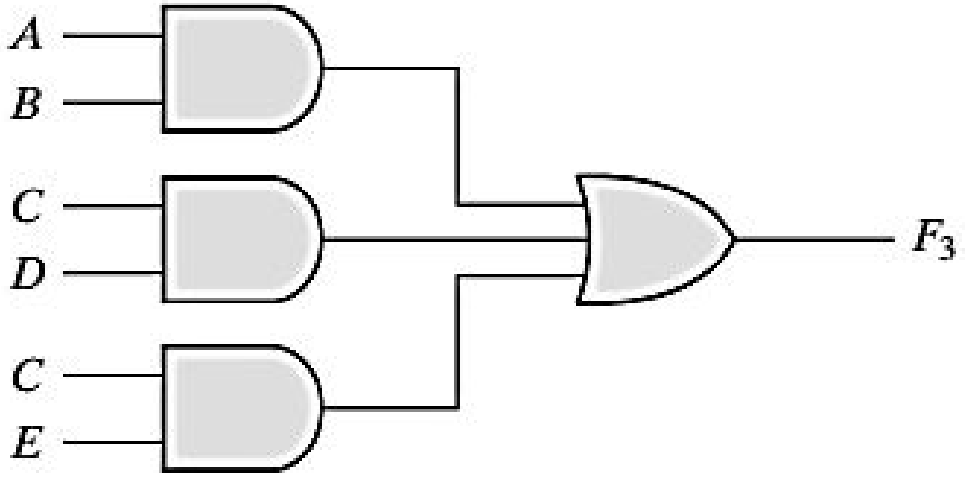
Nonstandard Form: $F_3 = AB + C(D + E)$

Standard Form: $F_3 = AB + C(D + E) = AB + CD + CE$

Standard Forms and Nonstandard Forms…



(a) $AB + C(D + E)$



(b) $AB + CD + CE$

Other Logic Operations

- For ***n*** number of Boolean variables-
  - 2^n number of rows of combinations can be produced of a truth table
  - In total 2^{2^n} number of different functions are possible (**How?**)
    - In figure, each column ($F_0 \sim F_{15}$) represents a truth table for one possible function
    - These functions also could be expressed as algebraic expressions (using SOP or POS)

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1

16 Boolean Functions: Algebraic Expressions

Boolean Functions	Operator Symbol	Name	Comments
$F_0 = 0$		Null	Binary constant 0
$F_1 = xy$	$x \cdot y$	AND	x and y
$F_2 = xy'$	x/y	Inhibition	x, but not y
$F_3 = x$		Transfer	x
$F_4 = x'y$	y/x	Inhibition	y, but not x
$F_5 = y$		Transfer	y
$F_6 = xy' + x'y$	$x \oplus y$	Exclusive-OR	x or y, but not both
$F_7 = x + y$	$x + y$	OR	x or y
$F_8 = (x + y)'$	$x \downarrow y$	NOR	Not-OR
$F_9 = xy + x'y'$	$(x \oplus y)'$	Equivalence	x equals y
$F_{10} = y'$	y'	Complement	Not y
$F_{11} = x + y'$	$x \subset y$	Implication	If y, then x
$F_{12} = x'$	x'	Complement	Not x
$F_{13} = x' + y$	$x \supset y$	Implication	If x, then y
$F_{14} = (xy)'$	$x \uparrow y$	NAND	Not-AND
$F_{15} = 1$		Identity	Binary constant 1

Other Logic Operations…

- 16 functions are subdivided into 3 categories:
  - 2 functions that produce **a constant** 0 or 1
    - Constant functions
  - 4 functions with **unary operations**
    - Complement and transfer (or buffer).
  - **10** functions with **binary operators** that define **eight** different operations
    - AND, OR, NAND, NOR, exclusive-OR (XOR), equivalence (XNOR), inhibition, and implication.

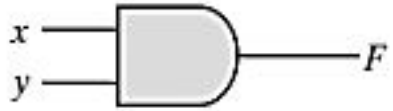
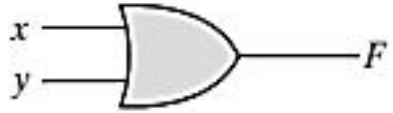
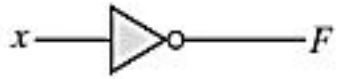
Digital Logic Gates

- Boolean functions are usually implemented with AND, OR, NOT gates
- Consideration to construct other logic gates:
  - **Feasibility** and **economy** of producing gates with physical components
  - The possibility of **extending the number of inputs** more than two
  - Basic properties of binary operator like **commutativity** and **associativity**
  - The ability to **implement Boolean function** alone or in conjunction with other gates

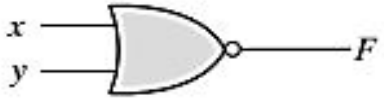
Digital Logic Gates: Candidates

- Among 16 functions from previous table with two variables
  - 2 functions are constants
  - 4 functions are repeated
  - **Only 10 functions** left to be considered as logic gates
- **Implication** and **Inhibition** don't follow the **associative** and **commutative** laws.
  - **Impractical** to use as standard logic gates
- So, others are used as standard gates in digital computer design
  - ***i.e.,*** Complement, transfer, AND, OR, NAND, NOR, XOR, XNOR

Digital Logic Gates: AND, OR, NOT, Buffer

Name	Graphic symbol	Algebraic function	Truth table
AND		$F = x \cdot y$	x y | F 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1
OR		$F = x + y$	x y | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1
Inverter		$F = x'$	x | F 0 | 1 1 | 0
Buffer		$F = x$	x | F 0 | 0 1 | 1

Digital Logic Gates: NAND, NOR, XOR, XNOR

Name	Graphic symbol	Algebraic function	Truth table
NAND		$F = (xy)'$	x y | F 0 0 | 1 0 1 | 1 1 0 | 1 1 1 | 0
NOR		$F = (x + y)'$	x y | F 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 0
Exclusive-OR (XOR)		$F = xy' + x'y$ $= x \oplus y$	x y | F 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0
Exclusive-NOR or equivalence		$F = xy + x'y'$ $= (x \oplus y)'$	x y | F 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1

Extension to Multiple Inputs: Problem

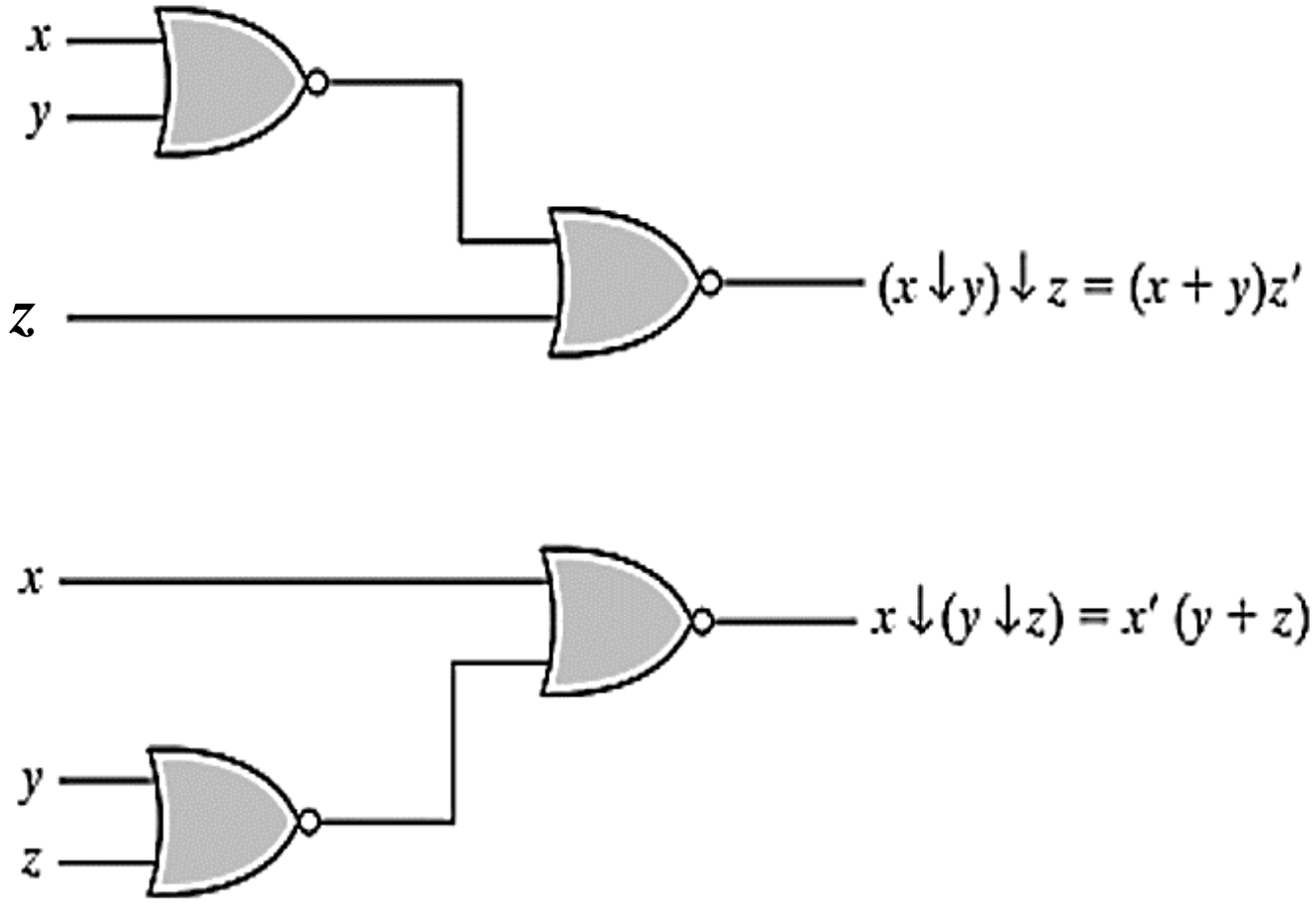
- A gate (other than inverter and buffer) can be extended to have multiple inputs if it follows **commutative** and **associative** laws (***e.g.,*** OR)

$$x + y = y + x \quad \text{(commutative)}$$
$$(x + y) + z = x + (y + z) = x + y + z \quad \text{(associative)}$$

- NAND and NOR are **commutative** and **not associative** so we can't extend them into multiple inputs in traditional way. (***e.g.,*** NOR)

$$(x \downarrow y) \downarrow z = [(x + y)' + z]' = (x + y)z' = xz' + yz'$$
$$x \downarrow (y \downarrow z) = [x + (y + z)']' = x'(y + z) = x'y + x'z$$

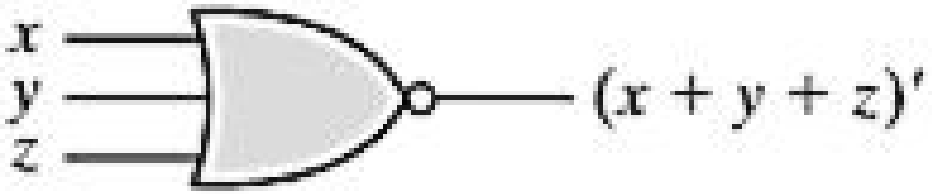
Extension to Multiple Inputs: Problem - NOR



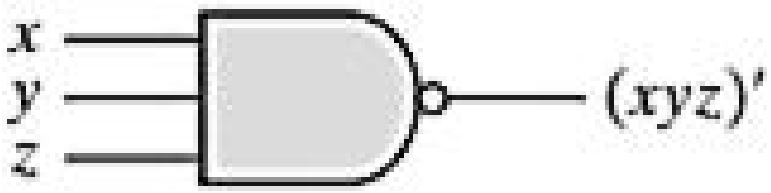
Extension to Multiple Inputs: Redefinition

- **To overcome this problem**, we should define the multiple NOR (or NAND) gate as a complemented OR (or AND) gate:

$$x \downarrow y \downarrow z = (x + y + z)'$$
$$x \uparrow y \uparrow z = (xyz)'$$



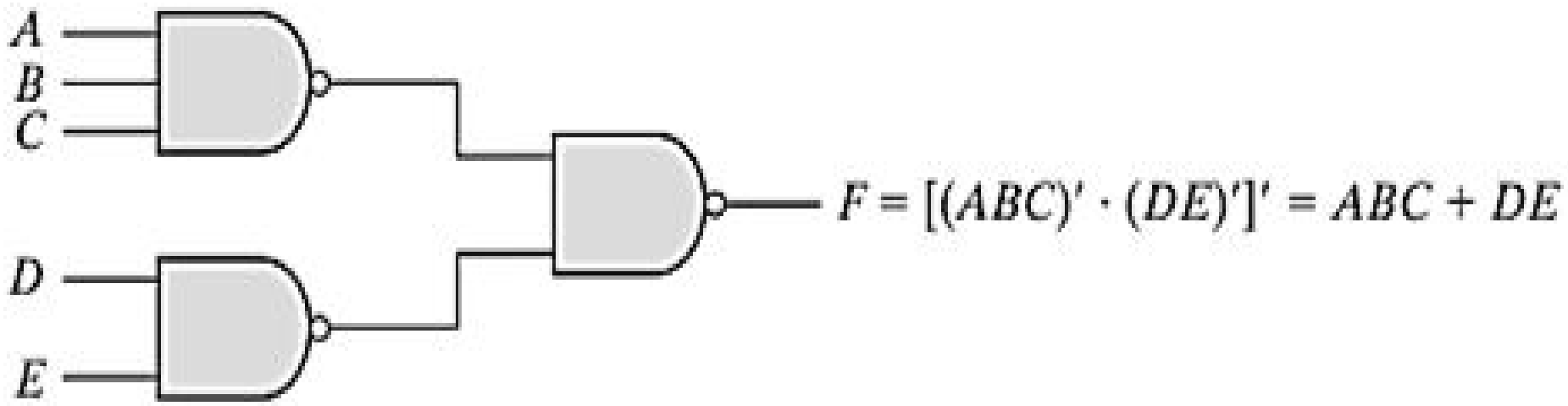
(a) 3-input NOR gate



(b) 3-input NAND gate

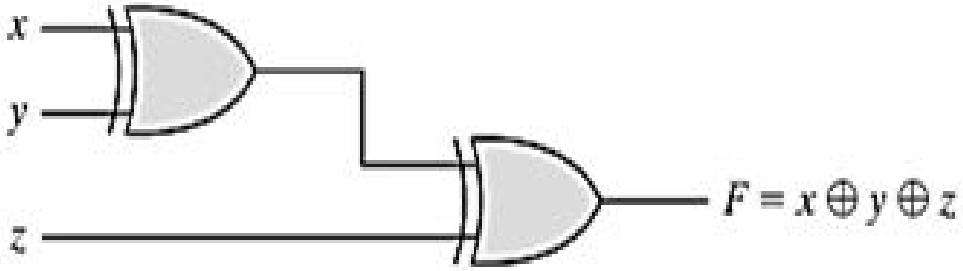
Extension to Multiple Inputs: Attention

- In **cascaded** NOR and NAND operations, we should use the **parenthesis** properly to make the sequence correct:

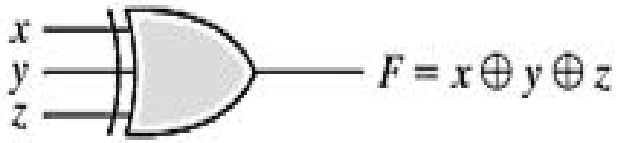


Extension to Multiple Inputs: XOR and XNOR

- Both are **associative** and **commutative**
  - But definition can be modified different way:
    - **XOR** – Odd function considering 1
    - **XNOR** – Even function considering 1, exclusively complement of XOR



(a) Using 2-input gates



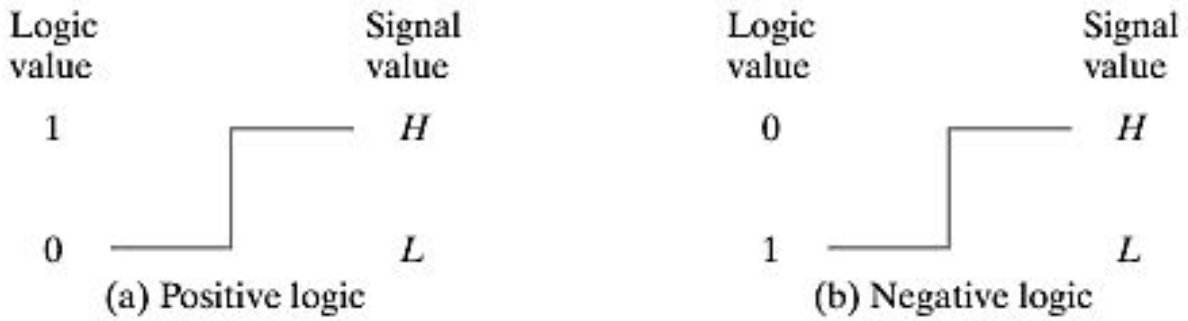
(b) 3-input gate

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

(c) Truth table

Positive and Negative Logic

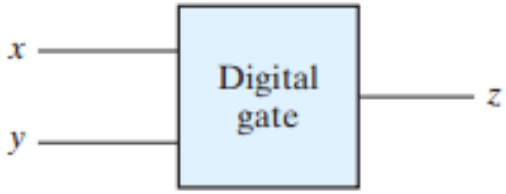
- Binary signal at inputs and outputs has one of two values
  - **Logic 1** and **Logic 0**
  - **Except during transition**
- There are two different assignments of analog signal levels (**H**igh and **L**ow) to these logic values
  - Assigning logic values to its **relative** amplitudes of signal levels – **sometimes both amplitudes may be considered as either positive or negative – So** actual amplitude is not that much informative
- If **High** (H) level represents logic 1 – **positive logic system**
- If **Low** (L) level represents logic 1 – **negative logic system**



Positive and Negative Logic…

x	y	z
L	L	L
L	H	L
H	L	L
H	H	H

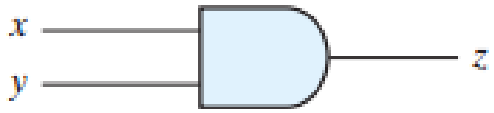
(a) Truth table with H and L



(b) Gate block diagram

x	y	z
0	0	0
0	1	0
1	0	0
1	1	1

(c) Truth table for positive logic



(d) Positive logic AND gate

x	y	z
1	1	1
1	0	1
0	1	1
0	0	0

(e) Truth table for negative logic

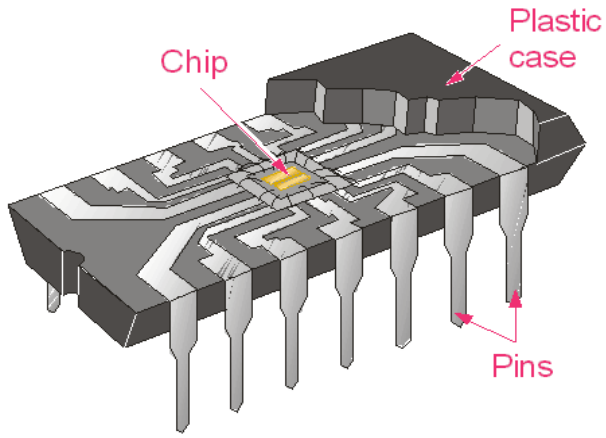
(f) Negative logic OR gate

Integrated Circuits

- Digital circuit constructed with **integrated circuit**
- **IC** – small die (dice) of **semiconductor crystal** named as **chip** based on what other components are embedded
    - **Components** of *Chip* – transistors, diodes, resistors, capacitors and so on
    - These components are interconnected inside
    - This chip is mounted (organized) on a **ceramic/plastic package** and connections are made of **external pin**
    - **Differ** from other **detachable electronic circuit** (with registers, capacitors)
    - **Designation of IC** printed on the surface

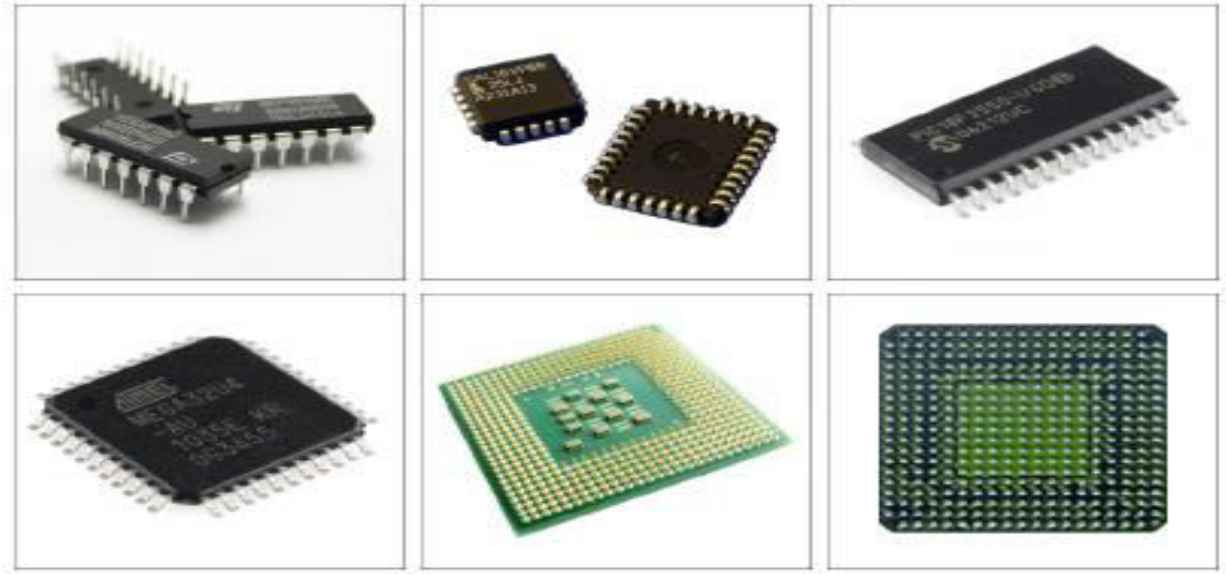
Integrated Circuits…

- **Advantages:**
  - Small in size
  - Cost effective
  - Reduced power consumption
  - High reliability against failure
- **Linear and Digital IC** – for **continuous** signal and **discrete** signal respectively from external sources
- **Two types** of packages:
  - Flat
  - Dual in Line



Types of ICs: Complexity

- Based on the **number of gates inside (complexity)** –
  - **SSI** – with several independent gates with their inputs and outputs (<10)
  - **MSI** – for specific elementary operations – decoder, encoder, adder, etc. (10~1000)
  - **LSI** – include digital system like processor, memory chip, etc. (>10,000)
  - **VLSI** – include complex microcomputer chip due to low cost and smaller in size (>1M)



Types of ICs: Technology

- **TTL** - transistor–transistor logic (standard)
- **ECL** - emitter-coupled logic (high speed)
- **MOS** - metal-oxide semiconductor (high component density)
- **CMOS** - complementary metal-oxide semiconductor (lower energy)

Digital logic family: Distinguishing Parameters

- **Parameters** in digital logic families:
  - Fan out = maximum number of output signals
  - Fan in = maximum number of input signals
  - Power dissipation
  - Propagation delay
  - Noise margin

Extension to Multiple Inputs…

Extension Multiple Inputs…